

내가 개발하려는 MCP 서버의 핵심 기능이 데이터 분석, 머신러닝, 혹은 수학적 연산과 관련되므로 python 을 써야 할거 같은데 추천하는 개발 플랫폼과 툴을 우선 알려줘.

데이터 분석 및 머신러닝 중심의 MCP 서버 개발을 결정하셨군요. 탁월한 선택입니다. 파이썬 생태계의 Pandas, Scikit-learn, PyTorch 등을 MCP 도구(Tool)로 연결하면 AI 모델이 직접 복잡한 연산을 수행하고 시각화 데이터까지 생성하게 만들 수 있습니다.

추천하는 개발 플랫폼과 툴셋을 단계별로 정리해 드립니다.

1. 핵심 개발 프레임워크: FastMCP

현재 파이썬으로 MCP 서버를 개발할 때 가장 추천되는 라이브러리는 **FastMCP** 입니다.

- **특징:** Anthropic 공식 SDK 를 기반으로 하며, FastAPI 와 유사하게 데코레이터(@mcp.tool())를 사용하여 함수를 바로 도구로 노출할 수 있습니다.
- **장점:** 복잡한 JSON-RPC 통신 규약을 직접 구현할 필요 없이, 파이썬 함수의 Type Hint 를 분석하여 모델이 이해할 수 있는 스키마를 자동으로 생성해 줍니다.

2. 권장 도구 및 라이브러리 조합 (Modern Tech Stack)

데이터 과학 프로젝트의 특성상 의존성 관리가 중요하므로 아래 조합을 추천합니다.

- **패키지 매니저: uv**
 - 기존 pip 나 conda 보다 압도적으로 빠르며, 프로젝트별 가상 환경 관리가 매우 간편합니다.
- **데이터 처리: Pandas, NumPy**
 - 설명이 필요 없는 데이터 분석의 표준입니다.
- **머신러닝/통계: Scikit-learn, Statsmodels**

- 가벼운 모델 추론이나 통계 분석 결과를 AI 에게 전달할 때 유용합니다.

- **디버깅 및 테스트: MCP Inspector**

- 서버를 실제 Claude Desktop 에 붙이기 전, 웹 인터페이스에서 도구가 잘 작동하는지 테스트할 수 있는 필수 도구입니다.

3. 개발 환경 구축 및 실행 예제

데이터 분석 기능을 포함한 MCP 서버의 기본 구조를 uv 기반으로 작성해 보겠습니다.

Step 1: 프로젝트 설정

```
# uv 설치 (이미 설치되어 있다면 건너뛰니다)
pip install uv

# 프로젝트 생성 및 이동
uv init data-analysis-mcp
cd data-analysis-mcp

# 필수 라이브러리 추가
uv add fastmcp pandas numpy scikit-learn
```

Step 2: 분석 서버 코드 작성 (server.py)

AI 가 데이터를 요약하고 기초적인 상관계수를 분석할 수 있도록 구성한 예시입니다.

```

import pandas as pd
import numpy as np
from fastmcp import FastMCP

# 1. 서버 초기화
mcp = FastMCP("DataAnalyzer")

# 2. 데이터 분석 도구 정의
@mcp.tool()
def analyze_csv_summary(file_path: str) -> str:
    """
    지정된 경로의 CSV 파일을 로드하여 기초 통계 요약 정보를 반환합니다.
    """
    try:
        df = pd.read_csv(file_path)
        summary = {
            "rows": len(df),
            "columns": list(df.columns),
            "stats": df.describe().to_dict(),
            "missing_values": df.isnull().sum().to_dict()
        }
        return f"데이터 요약 정보:\n{summary}"
    except Exception as e:
        return f"오류 발생: {str(e)}"

@mcp.tool()
def calculate_correlation(file_path: str, col1: str, col2: str) -> str:
    """
    두 컬럼 간의 피어슨 상관계수를 계산합니다.
    """
    try:
        df = pd.read_csv(file_path)
        corr = df[col1].corr(df[col2])
        return f"{col1}과 {col2}의 상관계수: {corr:.4f}"
    except Exception as e:
        return f"오류 발생: {str(e)}"

if __name__ == "__main__":
    # 3. 서버 실행
    mcp.run()

```

Step 3: 디버깅 및 실행

개발 중에는 아래 명령어를 통해 **MCP Inspector** 를 띄워 도구가 예상대로 출력되는지 확인하세요.

```
npx @modelcontextprotocol/inspector python server.py
```

4. 추천하는 배포 및 연동 방식

1. **로컬 테스트:** Claude Desktop 의 설정 파일(claude_desktop_config.json)에 위 파이썬 스크립트를 등록하여 사용합니다.
2. **컨테이너화:** 데이터 분석 환경은 라이브러리 버전 의존성이 강하므로, 최종적으로는 **Docker** 를 사용하여 환경을 격리하는 것이 운영 측면에서 훨씬 유리합니다.

이제 이 구조를 바탕으로 실제 어떤 데이터를 분석하거나 어떤 머신러닝 모델을 연동하고 싶으신지 알려주시면, 그에 맞춘 구체적인 도구(Tool) 설계 방법을 제안해 드릴 수 있습니다. 예를 들어, 특정 데이터베이스(SQL) 연동이 필요하신가요? 혹은 로컬 모델 추론 기능을 넣고 싶으신가요?

VS CODE 에서 개발도구와 라이브러를 설치하는 아주 기초적인 단계부터 차근차근 하나씩 알려줘. 내 WINDOWS PC 에는 VS CODE 와 Python 만 설치되어 있다고 가정하해

Windows PC 에 VS Code 와 Python 이 이미 설치되어 있다는 전제하에, MCP(Model Context Protocol) 서버 개발을 위한 환경 구축 과정을 가장 기초적인 단계부터 하나씩 설명해 드리겠습니다.

데이터 분석과 머신러닝 업무를 수행할 예정이므로, 환경을 독립적으로 관리하는 것이 매우 중요합니다. 다음 단계를 차근차근 따라와 주세요.

1. VS Code 필수 확장 프로그램 설치

VS Code 에서 파이썬 코드를 효율적으로 작성하고 실행하기 위해 먼저 도구를 보강해야 합니다.

1. VS Code 를 실행합니다.
2. 왼쪽 사이드바에서 블록 모양의 아이콘(Extensions)을 클릭합니다.
3. 검색창에 **Python** 을 입력하고, Microsoft 에서 만든 공식 Python 확장 프로그램을 설치합니다.
4. 이 프로그램을 설치하면 **Pylance**(코드 분석 도구)도 함께 설치되어 코드 자동 완성 기능을 제공합니다.

2. 프로젝트 폴더 생성 및 열기

1. 바탕화면이나 원하는 경로에 새 폴더를 만듭니다 (예: my-mcp-server).
2. VS Code 에서 **File > Open Folder** 를 선택하여 방금 만든 폴더를 엽니다.

3. 가상 환경(Virtual Environment) 구축

파이썬 프로젝트마다 별도의 공간을 만들어 라이브러리 충돌을 방지하는 단계입니다. 데이터 분석 프로젝트에서는 라이브러리 버전이 중요하므로 이 과정은 필수입니다.

1. VS Code 상단 메뉴에서 **Terminal > New Terminal** 을 클릭합니다.
2. 터미널 창(하단)에 다음 명령어를 입력하고 엔터를 누릅니다.

```
python -m venv .venv
```

3. 폴더 목록에 .venv 라는 폴더가 생겼는지 확인합니다.
4. 가상 환경 활성화:

- 터미널에 다음 명령어를 입력합니다.

```
.\.venv\Scripts\activate
```

- 활성화되면 터미널 경로 앞에 (.venv)라는 표시가 나타납니다.

4. 핵심 라이브러리 설치

이제 MCP 서버 개발과 데이터 분석에 필요한 라이브러리들을 설치합니다.
터미널에 다음 명령어를 입력하세요.

```
pip install fastmcp pandas scikit-learn
```

- **fastmcp**: MCP 서버를 쉽게 구축하게 해주는 프레임워크입니다.
- **pandas**: 데이터 로딩 및 분석을 위한 핵심 라이브러리입니다.
- **scikit-learn**: 머신러닝 데이터셋과 알고리즘을 제공합니다.

5. MCP 서버 코드 작성 및 데이터 로딩 예제

이제 실제로 동작하는 코드를 작성해 보겠습니다. 이 코드는 인터넷에서 유명한 'Iris(붓꽃)' 데이터셋을 불러와 기초 통계를 내주는 기능을 포함합니다.

1. VS Code 에서 새 파일을 만들고 이름을 main.py 로 저장합니다.
2. 아래 코드를 복사하여 붙여넣으세요.

```

import pandas as pd
from fastmcp import FastMCP
from sklearn.datasets import load_iris

# 1. MCP 서버 생성
mcp = FastMCP("DataAnalysisServer")

# 2. 데이터 분석 기능을 수행하는 도구(Tool) 정의
@mcp.tool()
def get_iris_summary() -> str:
    """
    Scikit-learn 라이브러리에서 Iris 데이터셋을 직접 로드하여
    데이터의 기초 통계 요약 정보를 반환합니다.
    """
    try:
        # 데이터셋 로드
        iris = load_iris()

        # 데이터프레임으로 변환 (데이터셋 구축 단계)
        df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
        df['target'] = iris.target

        # 데이터 전처리 및 분석 (기초 통계량 산출)
        summary = df.describe().to_string()

        return f"Iris 데이터셋 분석 결과:\n\n{summary}"
    except Exception as e:
        return f"데이터 로딩 중 오류 발생: {str(e)}"

@mcp.tool()
def read_remote_csv(url: str) -> str:
    """
    입력받은 URL의 CSV 데이터를 읽어와서 상위 5개 행을 보여줍니다.
    예시 URL: https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv
    """
    try:
        # 온라인 데이터셋 로딩
        df = pd.read_csv(url)

        # 데이터 확인 (전처리 단계의 시작)
        head_data = df.head().to_string()

        return f"원격 데이터 로드 성공. 상위 5행 데이터:\n\n{head_data}"
    except Exception as e:
        return f"CSV를 읽어오는 데 실패했습니다: {str(e)}"

if __name__ == "__main__":
    # 3. 서버 실행
    mcp.run()

```

6. 서버 테스트 (MCP Inspector)

작성한 서버가 실제로 AI와 통신할 준비가 되었는지 확인해야 합니다.

1. VS Code 터미널에서 다음 명령어를 실행합니다. (Node.js가 설치되어 있어야 하며, 없다면 python main.py로 실행만 확인합니다.)

```
npx @modelcontextprotocol/inspector python main.py
```

2. 이 명령어를 실행하면 브라우저에서 테스트 화면이 열리고, 작성한 get_iris_summary 도구가 정상적으로 동작하는지 버튼을 눌러 확인할 수 있습니다.

단계별 사고 과정 요약 (데이터 과학 관점)

1. **환경 격리**: 각 프로젝트가 독립적인 라이브러리 버전을 갖도록 가상 환경을 구축했습니다.
2. **의존성 관리**: 분석에 필요한 Pandas와 학습용 데이터셋을 제공하는 Scikit-learn을 설치했습니다.
3. **데이터 파이프라인 구축**: 코드를 통해 외부 URL이나 내장 라이브러리로부터 데이터를 로딩하는 로직을 먼저 작성했습니다.
4. **인터페이스 노출**: 작성된 파이썬 함수를 MCP의 @mcp.tool() 데코레이터를 통해 LLM(Claude 등)이 호출할 수 있는 형태로 변환했습니다.

이 단계까지 완료하셨다면 기본적인 서버 개발 준비가 끝난 것입니다. 이제 어떤 특정 데이터를 분석하거나 모델을 학습시키고 싶으신가요? 구체적인 분석 목표를 알려주시면 그에 맞는 코드를 추가로 작성해 드릴 수 있습니다.

니가 위에서 말한 uv 는 언제 설치되고 사용되는거야?

uv 는 최근 파이썬 생태계에서 매우 각광받고 있는 **초고속 패키지 및 프로젝트 관리 도구**입니다. 기존에 제가 설명해 드린 pip 와 venv 의 기능을 하나로 합쳐서 훨씬 더 빠르고 간편하게 수행한다고 이해하시면 됩니다.

데이터 과학이나 머신러닝 프로젝트에서는 관리해야 할 라이브러리가 많고 복잡하기 때문에, uv 를 사용하면 환경 구축 시간을 획기적으로 줄일 수 있습니다.

1. uv 는 언제 설치하고 사용하는가?

설정 시점 (설치)

- 가장 처음, 개발 환경을 구축하기 시작할 때 딱 한 번 설치합니다.
- Windows 의 경우 터미널(PowerShell)에서 간단한 명령어로 설치할 수 있습니다.

사용 시점 (워크플로우)

- 프로젝트 시작 시:** uv init 으로 프로젝트 폴더의 뼈대를 만들 때 사용합니다.
- 라이브러리 설치 시:** pip install 대신 uv add 를 사용하여 라이브러리를 추가합니다.
- 서버 실행 시:** 가상 환경을 일일이 활성화할 필요 없이 uv run 명령어로 즉시 코드를 실행할 때 사용합니다.

2. uv 를 활용한 MCP 서버 개발 단계 (단계별 안내)

기존의 pip 방식보다 훨씬 간결한 과정을 단계별로 설명해 드리겠습니다.

단계 1: uv 설치 (Windows PowerShell)

VS Code 의 터미널을 열고 다음 명령어를 입력하여 uv 를 설치합니다.

```
powershell -c "irmo https://astral.sh/uv/install.ps1 | iex"
```

설치가 완료된 후, 터미널을 껐다 다시 켜거나 refreshenv 명령어를 입력하여 uv 명령어가 인식되도록 합니다.

단계 2: 프로젝트 초기화

개발할 폴더로 이동하여 프로젝트를 생성합니다.

```
uv init data-analysis-mcp  
cd data-analysis-mcp
```

단계 3: 라이브러리 추가

MCP 개발과 데이터 분석에 필요한 라이브러리를 한 번에 추가합니다. uv 는 이 과정에서 자동으로 가상 환경을 생성하고 최적화된 속도로 설치를 진행합니다.

```
uv add fastmcp pandas scikit-learn
```

3. uv 환경에서 실행하는 데이터 분석 MCP 코드 예시

이제 uv 환경에서 바로 실행해 볼 수 있도록, 데이터를 로딩하고 전처리하는 과정을 포함한 코드를 작성해 보겠습니다. 이 코드는 잘 알려진 **California Housing**(캘리포니아 주택 가격) 데이터셋을 활용합니다.

파일명: app.py

```

# 1. MCP 서버 인스턴스 생성
mcp = FastMCP("HousingAnalysisServer")

def load_and_preprocess_data():
    """
    데이터 과학의 표준 단계인 데이터 로딩 및 전처리를 수행합니다.
    """
    # 데이터 로드
    housing = fetch_california_housing()
    df = pd.DataFrame(data=housing.data, columns=housing.feature_names)
    df['MedHouseVal'] = housing.target

    # 전처리: 결측치 확인 (이 데이터셋은 결측치가 없으나 일반적인 파이프라인 포함)
    if df.isnull().values.any():
        df = df.dropna()

    # 특징량 추출 및 스케일링 준비
    X = df.drop('MedHouseVal', axis=1)
    y = df['MedHouseVal']

    # 훈련/테스트 세트 분할
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    # 데이터 스케일링 (머신러닝 전처리 핵심 단계)
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)

    return df, X_train_scaled

```

```
# 2. MCP 도구 정의
```

```
@mcp.tool()
```

```
def get_housing_insight() -> str:
```

```
    """
```

```
    캘리포니아 주택 데이터를 로드하고 전처리한 후 기초 통계량과  
    데이터의 특성 (Feature)에 대한 인사이트를 반환합니다.
```

```
    """
```

```
    try:
```

```
        # 데이터 로딩 및 전처리 수행
```

```
        df, scaled_data = load_and_preprocess_data()
```

```
        # 분석 리포트 생성
```

```
        summary = df.describe().loc[['mean', 'std', 'min', 'max']]
```

```
        correlation = df.corr()['MedHouseVal'].sort_values(ascending=False)
```

```
        result = [
```

```
            "### 캘리포니아 주택 데이터 분석 요약 ###",
```

```
            f"전체 데이터 행 수: {len(df)}",
```

```
            "\n[주요 통계 정보]",
```

```
            summary.to_string(),
```

```
            "\n[주택 가격(목표 변수)과의 상관관계]",
```

```
            correlation.to_string()
```

```
        ]
```

```
        return "\n".join(result)
```

```
    except Exception as e:
```

```
        return f"데이터 분석 중 오류가 발생했습니다: {str(e)}"
```

```
if __name__ == "__main__":
```

```
    # 3. 서버 실행
```

```
    mcp.run()
```

4. uv 를 이용한 실행 방법

가상 환경을 직접 활성화할 필요 없이 아래 명령어로 즉시 실행하거나 디버깅할 수 있습니다.

- 서버 직접 실행:

```
uv run app.py
```

- MCP 인스펙터로 도구 테스트:

```
npx @modelcontextprotocol/inspector uv run app.py
```

요약 및 비교

구분	기존 방식 (pip + venv)	uv 방식
속도	라이브러리 설치 시 대기 시간이 김	Rust 기반으로 매우 빠름
관리	venv 활성화(activate)가 번거로움	uv run 으로 활성화 없이 즉시 실행
정리	requirements.txt 를 수동 관리	pyproject.toml 에 자동 기록 및 관리

이제 uv 를 설치하고 위 코드를 실행해 보시겠습니까? 설치 중에 어려움이 있다면 말씀해 주세요. 다음 단계로 실제 Claude Desktop 앱에 이 서버를 등록하는 방법을 도와드릴 수도 있습니다.

다시 정리할게 uv와 VS CODE와 PYTHON을 기반으로 MCP 서버를 개발하는 개발환경을 구축하는 단계를 처음부터 하나씩 차례대로 자세히 설명해 줘

Windows PC 환경에서 uv, VS Code, 그리고 Python을 사용하여 MCP(Model Context Protocol) 서버 개발 환경을 구축하는 과정을 단계별로 상세히 설명해 드리겠습니다.

1단계: uv 설치하기

uv는 파이썬 패키지 설치와 가상 환경 관리를 통합한 매우 빠른 도구입니다.

1. **터미널 실행:** Windows 작업 표시줄 검색창에 'PowerShell'을 입력하고 실행합니다.
2. **설치 명령어 입력:** 아래 명령어를 복사하여 PowerShell에 붙여넣고 엔터를 누릅니다.

```
powershell -c "irmo https://astral.sh/uv/install.ps1 | iex"
```

3. **설치 확인:** 설치가 완료되면 터미널을 종료했다가 다시 실행한 후, 아래 명령어를 입력하여 버전이 나오는지 확인합니다.

```
uv --version
```

2단계: 프로젝트 생성 및 초기화

개발할 서버의 전용 폴더를 만들고 프로젝트 구조를 잡는 단계입니다.

1. **프로젝트 폴더 생성:** 터미널에서 원하는 경로로 이동한 뒤 아래 명령어를 입력합니다.

```
mkdir my-mcp-analysis  
cd my-mcp-analysis
```

2. **uv 프로젝트 초기화:** 아래 명령어를 입력하면 pyproject.toml 파일과 기본적인 폴더 구조가 자동으로 생성됩니다.

```
uv init
```

3. **VS Code로 열기:** 현재 폴더를 VS Code에서 엽니다.

```
code .
```

3단계: 필수 라이브러리 설치

데이터 분석과 MCP 서버 구축에 필요한 도구들을 설치합니다. uv는 이 명령어를 실행할 때 자동으로 가상 환경을 생성하고 라이브러리를 관리합니다.

1. **라이브러리 추가:** VS Code의 내부 터미널(Ctrl + `)을 열고 아래 명령어를 입력합니다.

```
uv add fastmcp pandas scikit-learn numpy
```

- fastmcp: MCP 서버 개발을 위한 프레임워크입니다.
- pandas, numpy: 데이터 처리를 위한 필수 라이브러리입니다.
- scikit-learn: 머신러닝 데이터셋과 전처리 도구를 제공합니다.

4단계: VS Code 파이썬 인터프리터 설정

VS Code가 uv가 만든 가상 환경을 인식하도록 설정해야 합니다.

1. VS Code 오른쪽 하단이나 상단 명령 팔레트(Ctrl + Shift + P)에서 **Python: Select Interpreter**를 검색합니다.
2. 목록에서 프로젝트 폴더 내의 .venv/Scripts/python.exe 경로가 포함된 인터프리터를 선택합니다.

5단계: MCP 서버 코드 작성 (데이터 과학 파이프라인 포함)

이제 실제 데이터 분석 기능을 수행하는 서버 코드를 작성합니다. 데이터셋 로딩부터 전처리, 분석 결과 반환까지 포함된 전체 코드를 작성하겠습니다.

파일명: main.py

```

import pandas as pd
import numpy as np
from fastmcp import FastMCP
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# 1. MCP 서버 인스턴스 초기화
mcp = FastMCP("AdvancedDataAnalysisServer")

def process_wine_data():
    """
    와인 데이터셋을 로드하고 머신러닝을 위한 전처리를 수행하는 내부 함수입니다.
    """
    # 데이터 로드 (Scikit-Learn 라이브러리 활용)
    wine = load_wine()
    df = pd.DataFrame(data=wine.data, columns=wine.feature_names)
    df['target'] = wine.target

    # 전처리 1: 결측치 확인 및 제거 (이 데이터셋은 깨끗하지만 일반적인 단계로 추가)
    df = df.dropna()

    # 전처리 2: 특징(X)과 타겟(y) 분리
    X = df.drop('target', axis=1)
    y = df['target']

    # 전처리 3: 학습/테스트 데이터 분리
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    # 전처리 4: 데이터 스케일링 (표준화)
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    return df, X_train_scaled, wine.target_names

```



```

# 2. MCP 도구 정의
@mcp.tool()
def analyze_wine_quality() -> str:
    """
    와인 데이터셋의 화학적 성분을 분석하고 전처리된 데이터의 요약을 반환합니다.
    """
    try:
        # 데이터 구축 및 전처리 단계 실행
        df, scaled_x, target_names = process_wine_data()

        # 분석 수행
        summary_stats = df.describe().loc[['mean', 'std']]
        class_distribution = df['target'].value_counts().to_dict()

        # 결과 메시지 구성
        result = [
            "와인 데이터 분석 리포트",
            f"분석된 총 샘플 수: {len(df)}",
            f"와인 종류: {' '.join(target_names)}",
            "\n[화학적 성분 평균 및 표준편차]",
            summary_stats.to_string(),
            "\n[클래스별 데이터 분포]",
            str(class_distribution),
            "\n전처리 상태: 데이터 스케일링 및 학습/테스트 세트 분리 완료"
        ]

        return "\n".join(result)
    except Exception as e:
        return f"분석 중 오류가 발생했습니다: {str(e)}"

if __name__ == "__main__":
    # 3. MCP 서버 실행
    mcp.run()

```

6단계: 실행 및 테스트

개발한 서버가 정상적으로 동작하는지 확인하는 마지막 단계입니다.

1. **서버 실행 테스트:** 터미널에서 아래 명령어를 입력하여 코드가 오류 없이 실행되는지 확인합니다.

```
uv run main.py
```

2. **MCP Inspector를 통한 디버깅:**

MCP 서버는 표준 입출력(stdio)을 사용하므로, 전용 테스트 도구인 Inspector를 사용하면 편리합니다. (이 기능을 위해 Node.js가 설치되어 있어야 합니다.)

```
npx @modelcontextprotocol/inspector uv run main.py
```

3. **확인:** 실행된 Inspector 웹 화면에서 `analyze_wine_quality` 도구를 클릭하고 실행하여 분석 결과가 텍스트 형식으로 잘 나오는지 확인합니다.

개발 환경 구축 단계 정리

1. **uv 설치:** 전체 패키지 및 가상 환경 관리를 위한 기반을 마련했습니다.
2. **프로젝트 초기화:** uv init을 통해 표준화된 파이썬 프로젝트 구조를 생성했습니다.
3. **의존성 추가:** 데이터 과학에 필수적인 pandas, scikit-learn 등을 uv add로 안전하게 설치했습니다.
4. **코드 구현:** 데이터 로딩, 결측치 처리, 데이터 분할, 스케일링 등 머신러닝의 전 과정을 포함한 분석 도구를 작성했습니다.
5. **검증:** uv run과 inspector를 사용하여 서버가 실제 LLM 클라이언트와 통신할 준비가 되었는지 확인했습니다.

이 과정을 모두 마치셨다면, 이제 자신만의 고도화된 데이터 분석 도구를 MCP 서버에 추가해 나갈 수 있습니다. 추가로 연동하고 싶은 데이터 소스나 특정 머신러닝 모델이 있다면 말씀해 주십시오.